Practical 6

Aim: Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both. Write a program to solve the problem using process synchronization technique.

```
HP@DESKTOP-B4BD73N MINGW64 ~
$ gcc dining.c -o dining.exe -lpthread

HP@DESKTOP-B4BD73N MINGW64 ~
$ ./dining.exe
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 4 is eating (1)
Philosopher 3 is eating (1)
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 2 is eating (1)
Philosopher 3 finished eating
Philosopher 3 is thinking
Philosopher 1 is eating (1)
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 0 is eating (1)
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 4 is eating (2)
Philosopher 0 finished eating
Philosopher 0 is thinking
Philosopher 3 is eating (2)
Philosopher 4 finished eating
Philosopher 4 is thinking
Philosopher 3 finished eating
Philosopher 2 is eating (2)
Philosopher 3 is thinking
Philosopher 1 is eating (2)
Philosopher 2 finished eating
Philosopher 2 is thinking
Philosopher 0 is eating (2)
Philosopher 1 finished eating
Philosopher 1 is thinking
Philosopher 0 finished eating
Philosopher 4 is eating (3)
Philosopher 0 is thinking
Philosopher 4 finished eating
Philosopher 3 is eating (3)
Philosopher 4 has left the table
Philosopher 2 is eating (3)
Philosopher 3 finished eating
Philosopher 3 has left the table
Philosopher 1 is eating (3)
Philosopher 2 finished eating
Philosopher 2 has left the table
Philosopher 0 is eating (3)
Philosopher 1 finished eating
Philosopher 1 has left the table
Philosopher 1 finished eating
Philosopher 1 has left the table
Philosopher 0 finished eating
Philosopher 0 has left the table

All philosophers have finished dining.

HP@DESKTOP-B4BD73N MINGW64 ~
```

Code-

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
#define TIMES 3   // Number of times each philosopher eats

sem_t chopstick[N];
pthread_t philosopher[N];
sem_t room;

void* dine(void* num) {
    int id = *(int*)num;

    for(int i = 0; i < TIMES; i++) {

        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        sem_wait(&room);

        sem_wait(&chopstick[id]);
        sem_wait(&chopstick[(id+1)%N]);

        printf("Philosopher %d is eating (%d)\n", id, i+1);
        sleep(1);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id+1)%N]);

        sem_post(&room);

        printf("Philosopher %d finished eating\n", id);
    }

    printf("Philosopher %d has left the table\n", id);
    pthread_exit(NULL);
}

int main() {
    int i, phil_num[N];

    sem_init(&room, 0, N-1);

    for(i=0;i<N;i++)
        sem_init(&chopstick[i], 0, 1);

    for(i=0;i<N;i++) {
        phil_num[i]=i;
        pthread_create(&philosopher[i], NULL, dine, &phil_num[i]);
    }

    for(i=0;i<N;i++)
        pthread_join(philosopher[i], NULL);

    printf("\nAll philosophers have finished dining.\n");
    return 0;
}
```